

Conflict-Free Knowledge Graph Projection

A Three-Phase CRDT Pipeline for Multi-Agent Memory Systems

Author: Subrata Sadhu

Affiliation: Independent Researcher

Contact: sadhu.arka507@gmail.com

Date: 2026-06-26

Status: Draft v4 — Preprint

License: Text licensed under CC-BY-4.0. Code (`crdt_projection.py`) is part of the `agentic-memory` repository, licensed under Apache-2.0.

Reproducibility: All claims verified against `paper_pipeline/crdt_projection.py` and `paper_pipeline/test_pipeline.py` (51 tests, 100% pass). Run `python -m pytest test_pipeline.py` from the `paper_pipeline/` directory to reproduce results.

Artifacts: Reference implementation: `paper_pipeline/crdt_projection.py`. Test suite: `paper_pipeline/test_pipeline.py`. The CRDT projection layer is part of the local-first `agentic-memory` system. **AI Assistance Disclosure:** Conceptual drafting, literature review, and typesetting assistance used publicly available AI assistants and free-tier paid models. All technical claims, equations, and code references were verified manually against the codebase.

Abstract

Multi-agent systems that maintain a local-first knowledge graph face a specific synchronization problem: when two agents independently create what is semantically the same entity — the same name, the same type — they produce two distinct entity records, each with its own edges. Naive merge preserves both records, splitting the edge set between them and fragmenting the graph semantically.

We present a three-phase projection pipeline that resolves this problem without requiring a central coordinator or ID-at-creation protocol. The pipeline introduces a *durable redirect map* — a data structure that records which entity IDs were merged into which canonical winners, enabling pre-projection edge rewriting that eliminates orphan edges by construction. Phase 1 merges entity operations using a 2P-Set for membership and a Last-Writer-Wins Register per metadata field. Phase 2 groups by inception fingerprint (SHA-256 of content fields), selects a winner, and emits the redirect map. Phase 3 applies the redirect to edge endpoints before writing to the canonical table, ensuring the no-orphan invariant holds at write time. The convergence model assumes exact-broadcast delivery (all peers eventually receive the same operation set); delivery-order independence under partial replication is not modeled.

We evaluate the pipeline against three baselines: naive merge (ID-at-creation semantics, as in Yjs/Automerger), redirect-only (no orphan guard), and our full pipeline. On 5,000 concurrent entity ops with 50 distinct entities, naive merge produces 4,950 duplicates and 460 orphan edges; our pipeline deduplicates to 50 canonical entities with zero orphans at ~36–39% overhead — dominated by Phase 1 (94% of runtime), not by content-keyed dedup. The orphan guard achieves zero orphans by silently dropping edges to non-canonical entities — a deliberate tradeoff favoring invariant enforcement over edge preservation. The pipeline is backed by 86 tests (51 unit + 35 adversarial) and is a concrete instantiation of the CK-CRDT framework [companion paper].

1. Introduction

1.1 The Multi-Agent Memory Problem

Consider a local-first memory system shared between two agents — a laptop and a desktop — operating independently and synchronizing when connectivity permits. Each agent can create named entities (people, projects, concepts) and draw typed edges between them. Neither agent has a global view of the other’s writes at the time of creation, and neither can coordinate with the other before writing.

Under these conditions, concurrent entity creation is not a hypothetical edge case — it is the expected steady state. Two agents, both encountering the same person in different sessions, will both create an entity for that person. If they assign different internal IDs, the downstream consequences — split edge sets, fragmented queries, incorrect graph walks — propagate to every operation that references the entity.

1.2 Why Existing Approaches Fall Short

Mutexes and leader election require a central coordinator, which contradicts the local-first requirement. When both agents are offline, there is no coordinator to consult.

ID-at-creation protocols (as used in Yjs, Automerge, and Loro) assign globally unique identifiers at insertion time, so concurrent creation of the same concept produces two distinct nodes that must be reconciled later. These systems do not collapse duplicates — they accept proliferation and leave deduplication to the application layer.

Post-hoc cleanup without redirect mapping (as in content-addressed systems like IPFS [9] and Syncthing) detects orphan edges after the fact but does not rewrite them. The application layer must handle dangling references.

No existing system resolves the problem at projection time by emitting an explicit redirect map that rewrites edge references before they enter the canonical table. The approaches we surveyed fall into two categories: some prevent concurrent duplicate creation by design (Yjs [4], Automerge [5], and Loro [6] assign globally unique IDs at creation time), while others detect and handle the consequences after the fact (content-addressed systems such as IPFS [9] and Syncthing use tombstone invalidation and garbage collection). Neither strategy produces a canonical graph with a no-orphan guarantee at the moment of projection.

1.3 Our Approach

We treat the knowledge graph as having two layers: an operation log layer where every peer writes independently, and a canonical projection layer where the merge happens. The projection layer runs a three-phase pipeline:

1. **Entity merge** — apply 2P-Set membership and LWW per field to resolve which entities are alive and what their current metadata is.
2. **Canonical entity resolution** — group surviving entities by their inception fingerprint (§2.5), select a winner per group, and produce a redirect map.
3. **Edge projection with redirect** — before writing edges to the canonical table, rewrite any edge endpoint that appears in the redirect map.

The result is a projection with a strong structural invariant: every edge in the canonical table points to an entity that also exists in the canonical table. We call this the **no-orphan invariant**. The convergence model assumes exact-broadcast delivery: all peers eventually receive the same operation set. Delivery-order independence under partial replication is not modeled.

1.4 Contribution

We contribute:

- **A durable redirect projection as a composition of existing CRDT patterns.** Prior systems either don’t deduplicate (Yjs, Automerge) or deduplicate post-hoc (content-addressed storage). The pipeline composes a 2P-Set, LWW-Registers, and a fingerprint-keyed redirect map into a durable projection step that rewrites edge endpoints before they enter the canonical table — eliminating orphan edges by construction rather than detection.
- The three-phase projection pipeline as a concrete CK-CRDT instantiation, with a standalone reference implementation and 86 tests (51 unit + 35 adversarial).
- Two no-orphan invariants: Theorem 1 (under a stated precondition, via the redirect map) and Corollary 1 (unconditional, via the orphan guard).
- The inception fingerprint as a deterministic entity-identity function, with Theorem 2 proving that $\max(\text{id})$ is the unique monotone selection rule for fingerprint-group dedup.
- Theorem 3 (convergence): the pipeline is deterministic over the operation set, proved via commutativity, associativity, and idempotence of each phase.
- Theorem 4 (lossless projection): the canonical state retains exactly the hierarchy of equivalence classes (entity-LWW nested within fingerprint groups, edge-LWW disjoint) plus winner designation, with a nested cascade where fingerprint grouping subsumes LWW. Corollary 2 bounds the information available to an adversary with access only to the canonical state.
- **Quantitative baseline comparison** (Table 7): on 5,000 concurrent entity ops, naive merge (ID-at-creation) produces 4,950 duplicates and 460 orphan edges; our pipeline deduplicates to 50 canonical entities with zero orphans at ~36–39% overhead.

2. Background and Related Work

2.1 CRDT Foundations

Conflict-free Replicated Data Types (CRDTs) are data structures that can be replicated across multiple nodes, updated independently, and merged without coordination, converging to a consistent state [1]. Convergence is guaranteed by satisfying four properties: commutativity, associativity, idempotence, and a well-defined merge result.

Two CRDT mechanisms are central to our pipeline:

2P-Set (Two-Phase Set): A set CRDT that tracks membership through two internal sets: an “add” set and a remove set. An element is a member if and only if it appears in the add set and not in the remove set. On concurrent add/remove, the add wins. We apply a strict variant of this rule to entity lifecycle: an entity is tombstoned if *any* remove operation causally dominates *any* add operation. Equivalently, an entity exists only if *every* add operation survives — no remove dominates it. In practice, each `entity_id` has exactly one add operation (agents create entities with unique IDs), so this is equivalent to the standard 2P-Set add-wins rule.

LWW-Register (Last-Writer-Wins Register): A register CRDT that resolves concurrent writes by selecting the one with the highest timestamp (or, more precisely, the one that causally follows the other via a version vector). Ties are broken by a deterministic tiebreaker. We apply one LWW-Register per metadata field — name, entity_type, description — so that concurrent writes to different fields do not interfere.

Version vectors: A causal-clock mechanism that tracks, for each peer, how many operations that peer has contributed. Two version vectors can be compared to determine whether one operation causally follows another, or whether they are concurrent. Version vectors are the foundation of the LWW tiebreaker and the add-wins rule in our 2P-Set.

2.2 Knowledge Graph Synchronization

Synchronizing knowledge graphs across distributed peers has been addressed in several contexts. RDF stores that apply CRDT semantics at the triple level merge triples individually; when two peers add triples with the same subject URI concurrently, the merge preserves both triples rather than collapsing them into a single subject node. Collaborative whiteboard tools (Figma, FigJam, Miro) handle concurrent node creation by assigning globally unique identifiers at insertion time. Concurrent “same-name” nodes remain distinct in the graph, each with their own connector edges — the duplication is accepted rather than resolved.

Entity deduplication in data integration is a well-studied problem [2, 3]. Probabilistic record linkage and deterministic blocking strategies are used in data lakes and master data management. The ASIF framework (2023) extends a G-Counter CRDT with per-field LWW-Register semantics for entity reconciliation in heterogeneous big-data integration, but addresses entity attribute consistency rather than graph edge integrity.

2.3 Multi-Agent Systems

Multi-agent systems that construct shared knowledge graphs encounter the same concurrent-creation problem at the agent level. Content-addressed systems (IPFS [9], Syncthing) handle edges pointing to deleted nodes through tombstone invalidation and garbage collection, detecting orphans after the write has already propagated. Our approach resolves the same problem at projection time by issuing a directed rewrite, eliminating the window of inconsistency entirely.

2.4 CRDT Composition

Composing multiple CRDTs requires that their merge operations preserve convergence when combined. Delta-based CRDTs (Loro, Automerge) apply operations as composable patches, where the patch application itself must satisfy CRDT properties. Our pipeline composes its three phases — entity merge, dedup, edge redirect — as sequential pure functions. The redirect step (rewriting edge endpoint IDs) is not idempotent in isolation, but becomes idempotent within the pipeline because the redirect map is derived from the Phase 1 result, which converges under CRDT merge semantics. We present the full convergence argument in Section 5.

2.5 Inception-Fingerprint Identity

The (name, entity_type) business key used in Phase 2 (v2) is too coarse for two cases: **homonyms** — two distinct entities sharing the same name and type (e.g., “Alice” the lawyer and

“Alice” the chef) — and **migration ID reassignment** — where a re-imported entity receives a new local ID that allows an outsider with a higher ID to steal the canonical slot.

Fingerprint formula. We replace the `(name, entity_type)` grouping key with a fingerprint computed from the distinguishing context available at entity creation time:

```
fingerprint = SHA-256(canonicalize(name) | canonicalize(entity_type) | canonicalize(description))
```

where `canonicalize(s)` lowercases, strips leading/trailing whitespace, and collapses internal whitespace to single spaces.

Why description as the discriminator. The `description` field captures the distinguishing context available at extraction time: “corporate lawyer at Skadden” vs “executive chef at Le Bernardin”. Alternative discriminators fail: `peer_id` would prevent cross-peer collapse (same entity created by different agents would never merge); `timestamp` would make every entity unique.

Immutability. The fingerprint is computed at inception and never recomputed. Changing an entity’s description after creation is a metadata update (Phase 1 LWW), not an identity change. This ensures identity continuity across enrichment cycles.

Backwards compatibility. When `description=""`, the fingerprint reduces to `SHA-256(name|type|)`, which produces identical grouping to the old `(name, entity_type)` key. All pre-fingerprint tests (29 of the current 51) pass without modification.

Strict at write path, permissive at backfill. New writes always compute and store the fingerprint at creation time. Legacy entities without fingerprints are backfilled retroactively from current `(name, type, description)` — an approximation (inception metadata may differ from current metadata) that always produces a value and is strictly better than the old approach which ignores description entirely.

3. Problem Statement

3.1 Concurrent Entity Creation

We formalize the problem by example. Consider two agents, A and B, each with an independent view of a knowledge graph.

Agent A creates an entity representing a person named “alice” and assigns it internal ID 42. It then creates an edge connecting entity 42 to an existing entity 15 (representing “bob”) with the relation `collaborates_with`.

Agent B, operating independently and without knowledge of A’s write, creates an entity also named “alice” of the same type “person” and assigns it internal ID 99. It creates an edge connecting 99 to entity 23 (representing “charlie”) with the same relation.

Both writes are valid. Neither agent did anything wrong. The problem arises when the two operation logs are merged.

3.2 Naive Merge: What Goes Wrong

A naive merge that simply unions the two operation logs and applies them without deduplication produces:

- **kg_entities:** two rows — {id=42, name='alice', type='person'} and {id=99, name='alice', type='person'}
- **kg_edges:** two edges — {source=42, target=15} and {source=99, target=23}

The edge set is split. Downstream consumers — retrieval by entity name, graph traversal, importance scoring — must now deal with two “alice” nodes. Any query that finds “alice” must be told to follow both IDs. Any edge added to “alice” after the merge must be duplicated across both rows. This is not a theoretical concern: it manifests as missed relationships, duplicate scoring, and incorrect graph walks in any system that assumes entity ID uniqueness.

3.3 Why a Mutex Doesn’t Help

A naive solution is to require agents to acquire a lock before creating an entity. In a local-first system, this is impossible: the agents may be offline when the concurrent creation occurs, and there is no central lock manager available. Even in an online setting, lock acquisition introduces a coordination point that degrades to the centralized system the local-first design is trying to avoid.

3.4 Why ID-at-Creation Doesn’t Help

Assigning globally unique IDs at creation time (UUIDs, random nonces) prevents ID collision but does not solve the semantic problem. Two agents still create two entities with the same name and type. The only difference is that both survive the merge — they are now two permanent nodes in the graph, distinguished only by their IDs. The downstream fragmentation problem is unchanged; the only difference is that it cannot be automatically repaired because there is no principled basis for declaring which ID is “the real one.”

3.5 Why Post-Hoc Cleanup Is Insufficient

Detecting orphan edges after merge and deleting or invalidating them leaves a window during which the canonical table is inconsistent. Any query issued in that window sees edges pointing to entities that do not yet exist in **kg_entities**. For interactive systems where agents issue queries in real time, that window is intolerable. The correction must be atomic with the projection itself.

3.6 What We Need

We need a merge procedure that:

1. Produces exactly one canonical row per real-world entity.
2. Ensures every projected edge references only canonical entity IDs.
3. Requires no coordination between agents at write time.
4. Provides an auditable record of which IDs were merged and why.
5. Converges: all peers applying the same operation log reach the same canonical state regardless of order.

The three-phase projection pipeline satisfies all five requirements.

4. The Three-Phase Projection Pipeline

4.1 Overview

The pipeline takes as input two operation log tables — `kg_entity_crdt` and `kg_edge_crdt` — and produces as output a single row per canonical entity in `kg_entities` and a set of edges in `kg_edges` that reference only canonical entity IDs. The operation logs are append-only and written by multiple peers without coordination.

flowchart TD

```
A[kg_entity_crdt] --> B[Phase 1: merge]
B --> C[merged_state]
C --> D[Phase 2: dedup]
D --> E[merged_state<br/>winners only]
D --> F[redirects<br/>{loser→winner}]
E --> G[_apply_entities]
F --> G
G --> H[kg_entities]
F --> I[Phase 3: redirect]
J[kg_edge_crdt] --> K[merge edges]
K --> I
I --> L[_apply_edges]
L --> M[kg_edges]
```

Figure 1: Data flow through the three-phase projection pipeline.

4.2 Phase 1: Entity CRDT Merge

Purpose: Resolve which entities are alive and what their current metadata is, using only the operation log.

Mechanism: Two rules applied per entity ID:

Membership (2P-Set): An entity is tombstoned if any “remove” operation causally dominates any “add” operation for the same entity_id. Equivalently, an entity exists only if every add operation is not dominated by any remove. On concurrent add/remove, the add wins. We apply this strict variant (tombstoned iff there exists a remove dominating some add) rather than the standard 2P-Set (there exists an add surviving all removes) because each entity_id has exactly one add operation in practice, making the two rules equivalent.

Metadata (LWW per field): For each metadata field — name, entity_type, description — the value from the operation with the highest version vector wins. Ties are broken by wall-clock timestamp, then by agent ID in lexicographic ascending order. This tiebreaker makes the merge deterministic across all replicas regardless of operation arrival order.

Key property: The merge is a pure function of the operation set. Given the same set of operations in any order, all replicas compute the same merged state. This is the convergence guarantee that makes the rest of the pipeline correct.

Pseudocode:

```
merge_entity_ops(ops):
    group ops by entity_id
```

```

for each entity_id:
    # sort by (timestamp, serialized VV) ONLY for deterministic iteration
    sort ops by (timestamp, serialized version_vector)
    adds      = ops where op == "add"
    removes   = ops where op == "remove"
    if no adds: skip (never created)
    is_tombstoned = any remove dominates any add
    if is_tombstoned: skip
    for each metadata field:
        # LWW: vv_dominates partial order is PRIMARY. Only truly concurrent
        # ops (neither dominates) fall back to (timestamp desc, agent_id asc).
        winner = adds[0]
        for candidate in adds[1:]:
            if vv_dominates(candidate.vv, winner.vv):
                winner = candidate
            elif not vv_dominates(winner.vv, candidate.vv):
                if candidate.ts > winner.ts or (
                    candidate.ts == winner.ts
                    and candidate.agent_id < winner.agent_id):
                    winner = candidate
        merged[entity_id][field] = winner.field
    merged[entity_id].tombstone = False
return merged

```

4.3 Phase 2: Canonical Entity Resolution

Purpose: If two distinct entity IDs represent the same real-world concept, pick one as the canonical representative and record the relationship between them.

Mechanism: After Phase 1, the merged state is keyed by entity ID. We group entries by their inception fingerprint (§2.5). Each fingerprint identifies one logical entity; entities with the same fingerprint are concurrent representations of the same real-world concept and collapse via `max(entity_id)`. Entities with different fingerprints are distinct, even if they share the same `(name, entity_type)` pair — this solves the homonym problem (§2.5).

For each fingerprint group:

- If the group has exactly one entry, it is already canonical; no action taken.
- If the group has multiple entries, we select the entry with the highest entity ID as the LWW winner. The deterministic tiebreaker here is entity ID value (higher wins), which is stable and does not require additional version-vector negotiation.

Legacy entities without fingerprints are backfilled at projection time: the fingerprint is computed from current `(name, type, description)` and stored in the merged state. This is an approximation but always produces a value.

The output of Phase 2 is two structures: - `merged_state`: the subset of the Phase 1 result containing only winner entries. - `redirects`: a map `{loser_id: winner_id}` for every entity that was merged away.

Key property: The redirect map is a pure function of the Phase 1 merged state. It is deterministic:

the same merged state always produces the same redirect map. This is what makes Phase 3 idempotent.

Pseudocode:

```
entity_dedup_via_crdt(merged_state):
    by_fingerprint = group merged_state entries by fingerprint
    for each fingerprint group:
        if len(ids) == 1:
            keep the entry, no redirects
        else:
            winner = max(ids) # highest entity_id = deterministic LWW
            merged_state = {winner: state[winner]}
            for loser in ids where loser != winner:
                redirects[loser] = winner
    return {merged_state, redirects}
```

4.4 Phase 3: Edge Projection with Redirect

Purpose: Before writing edges to the canonical `kg_edges` table, rewrite any edge endpoint that was merged away in Phase 2.

Mechanism: For each edge in the merged edge state, check whether `source_id` or `target_id` appears as a key in the redirect map. If so, replace it with the corresponding canonical winner ID. Edges whose endpoints are not in the redirect map pass through unchanged.

Before the redirect is applied, the edge CRDT log is merged using a LWW rule with the same comparison semantics as entity fields: the operation whose version vector causally dominates the others wins (`vv_dominates` partial order); when two ops are truly concurrent (neither dominates), ties are broken by higher timestamp, then by lexicographically *lowest* agent ID. This is identical to the entity-field tiebreaker (§4.2) and produces a single resolved edge per `edge_id` before the redirect pass.

An earlier version of this paper described the edge merge as using the *sum* of version-vector components as the primary sort key (`vv_sum`). That was incorrect: `vv_sum` conflates concurrent operations from different peers (e.g. `{A:3,B:0}` and `{A:0,B:3}` both sum to 3) and loses causal ordering guarantees. The paper and reference implementation are corrected to use `vv_dominates`, matching the production implementation in `kg/kg_crdt.py`¹.

Key property: Entity-field and edge merges use *identical* tiebreaker semantics (`vv_dominates` primary; concurrent $\rightarrow$ timestamp desc, then lowest agent ID asc). This uniformity is simpler than asymmetric biasing and is still fully deterministic: every replica applying the same operation set computes the same winner regardless of arrival order.

Key invariant (the paper’s central claim):

No-orphan invariant: After the three-phase pipeline completes, every `source_id` and every `target_id` in `kg_edges` references an `entity_id` that exists in `kg_entities`.

¹**Corrigendum (2026-07-16).** The `vv_sum` $\rightarrow$ `vv_dominates` correction affects only the edge merge tiebreaker. The entity merge was always correct (`vv_dominates` was used for entity fields from the start). The `vv_sum` function is retained in the codebase for diagnostic purposes only and is never called in any merge path.

The invariant is **prevented by construction, not merely detected after the fact**. Two structural guarantees enforce it at write time:

1. **Phase 3 endpoint rewrite.** When the precondition (no edge references a tombstoned entity ID; see Theorem 1 in §5.2 and the tombstone caveat in §7.1.1) is satisfied, Phase 3 rewrites edge endpoints *before* the edge is written to the canonical table. If an endpoint was merged away, the redirect map maps it to the winner; if it was not merged away, it was already canonical and is passed through. There is no path by which a non-canonical ID can reach the canonical edge table. (Theorem 1’s precondition is necessary for the redirect map; Corollary 1 shows the orphan guard eliminates the precondition at the system level.)
2. **Write-time canonicalization (resolve_edge_endpoints).** Both the CRDT projection path (`apply_edge_crdt_to_db`) and the primary direct edge-insert path (`kg_db._upsert_edge`) canonicalize every endpoint through `kg_entity_redirect` immediately before the `INSERT`. This means a merged-away loser ID cannot be persisted as an edge endpoint regardless of which code path performs the write — even if the in-memory `redirects` map is incomplete or a concurrent writer bypasses Phase 2. This also resolves the concurrent edge write scenario (§5.4, issue 3): if edge $X \rightarrow Y$ is written after Y was merged into Z , the durable redirect table maps $Y \rightarrow Z$ at write time, so the edge is rewritten to $X \rightarrow Z$ rather than being dropped or left as an orphan.

`verify_crdt_consistency` remains as a defense-in-depth post-condition assertion (it runs after every projection and raises if violated), but it is no longer the *only* guarantee: the invariant holds by construction at the moment of write. Theorem 1 (§5.2) proves the common case via the redirect map; Corollary 1 (§5.2) strengthens this to an unconditional guarantee via the orphan guard, which drops edges referencing tombstoned or never-created entities.

Idempotence: Applying the redirect map twice yields the same result as applying it once. After the first application, all loser IDs in edge endpoints have been replaced by winner IDs. Winner IDs do not appear as keys in the redirect map (by construction of Phase 2). Therefore, the second application finds no matches and makes no changes.

Pseudocode:

```

redirect_edge_ids(edge_state, redirects):
    if redirects is empty: return edge_state
    for each edge in edge_state:
        new_src = redirects.get(edge.source_id, edge.source_id)
        new_tgt = redirects.get(edge.target_id, edge.target_id)
        remapped[edge] = {edge with source_id=new_src, target_id=new_tgt}
    return remapped

# Orphan guard (defense-in-depth, see Corollary 1):
canonical_ids = set of entity IDs in canonical state
for each edge in remapped:
    if source_id not in canonical_ids or target_id not in canonical_ids:
        drop edge # handles tombstoned / never-created entities

```

4.5 End-to-End Walkthrough

The following scenario demonstrates the pipeline in operation. It is the same scenario exercised by the test suite in `paper_pipeline/test_pipeline.py`.

Setup. The system contains two pre-existing entities: entity 15 (“bob”) and entity 23 (“charlie”). These are already in `kg_entities` and have no edges between them.

Agent A creates entity 42 (“alice”, type “person”) with timestamp 100.0, and adds edge (42 → 15) with relation `collaborates_with`.

Agent B creates entity 99 (“alice”, type “person”) with timestamp 200.0, and adds edge (99 → 23) with the same relation.

Both operations are written to the CRDT log tables without coordination.

Phase 1 result. The entity merge processes all four entity operations (15, 23, 42, 99). Each has a valid “add” operation not preceded by a remove, so all four survive:

```
merged entity IDs: [15, 23, 42, 99]
entity 15: bob / person
entity 23: charlie / person
entity 42: alice / person
entity 99: alice / person
```

Both 42 and 99 survive because each has an add operation with no causally-dominating remove. The 2P-Set membership rule does not know they are semantically the same entity — that is Phase 2’s job.

Phase 2 result. Grouping by fingerprint (all four entities have `description=""`, so fingerprints are identical for same (name, type) pairs):

- (“bob”, “person”) → group size 1, no dedup → entity 15 kept.
- (“charlie”, “person”) → group size 1, entity 23 kept.
- (“alice”, “person”) → group size 2 (42, 99). Winner = max(42, 99) = 99.

```
canonical entities: [15, 23, 99]
redirect map: {42: 99}
```

Entity 42 is merged away. Every downstream reference to 42 must now point to 99.

Phase 3 result. Merging edges from the edge operation log produces two edges:

```
before redirect: [(99 → 23), (42 → 15)]
```

Applying the redirect map {42: 99}:

- Edge (99 → 23): neither endpoint is in redirects → unchanged.
- Edge (42 → 15): source 42 is in redirects → rewritten to (99 → 15).

```
after redirect: [(99 → 23), (99 → 15)]
```

Final state:

```
kg_entities: 15(bob), 23(charlie), 99(alice)    - 3 rows
kg_edges:    (99 → 23) collaborates_with
              (99 → 15) collaborates_with      - 2 rows
```

orphan edges: 0

Table 1 summarizes the state of the system after each phase.

Phase	kg_entities	redirects	kg_edges	Orphans
Before pipeline (CRDT logs)	{42, 99} (duplicates in entity log)	—	{(42→15), (99→23)} (in edge log)	—
After Phase 1	{15, 23, 42, 99}	—	—	—
After Phase 2	{15, 23, 99}	{42: 99}	—	—
After Phase 3	{15, 23, 99}	{42: 99}	{(99→15), (99→23)}	0

Table 1: System state after each phase of the pipeline. The first row shows the contents of the CRDT operation logs before any phase executes; the **kg_entities** and **kg_edges** columns for that row describe which entity and edge IDs are present in the logs, not yet written to the canonical tables. The no-orphan invariant holds after Phase 3.

Both edges now point to entity 99 (“alice”), the canonical winner. Entity 15 (“bob”) and entity 23 (“charlie”) are untouched. The no-orphan invariant holds.

4.6 Why the Ordering Matters

The three phases are not interchangeable. The critical ordering constraints are:

1. **Entity merge before dedup:** Phase 2 requires the fully-resolved entity metadata (name, type) to group by the business key. Running dedup before merge would use stale or conflicting field values, producing a different and potentially incorrect redirect map.
2. **Dedup before edge application:** Phase 3 requires the redirect map as input. Running edge projection before dedup would write edges pointing to IDs that may later be merged away, violating the invariant.
3. **Redirect before edge canonical write:** Applying the redirect map after writing to **kg_edges** is too late — the invariant is already violated. The redirect must be applied to the edge state in memory before the canonical write.

These ordering constraints are what make the pipeline a *pipeline* rather than three independent operations.

5. Formalization

5.1 Definitions

Let: - $\mathcal{P}$ be the set of peers (agents) in the system. - $\mathcal{O}_E$ be the set of all entity CRDT operations, each of the form `add(entity_id, peer, vv, name, type, ts)` or `remove(entity_id, peer, vv, ts)`. - $\mathcal{O}_{Ev}$ be the set of all edge CRDT operations, each of the form `add_edge(edge_id, src, tgt, relation, weight, peer, vv, ts)`. - $\mathcal{VV}$ be the set of version vectors, where $\mathcal{VV}(a,$

b) means vector a causally dominates vector b . - $LWW(a, b)$ be the Last-Writer-Wins comparison: a wins if $VV(a.vv, b.vv)$, or if concurrent and $a.ts > b.ts$, or if tie and $a.peer \leq b.peer$ lexicographically.

5.2 No-Orphan Invariant

Invariant 1 (Entity existence): After Phase 1 merge, an entity e exists in `merged_state` if and only if for every add operation o in `O_E` with $o.entity_id = e.id$, no remove operation r in `O_E` with $r.entity_id = e.id$ satisfies $vv_dominates(r.vv, o.vv)$. In other words, the entity is tombstoned iff there exists a remove that dominates any add.

Invariant 2 (Canonical uniqueness): After Phase 2, for any two distinct canonical entities $e1, e2$ in `merged_state`, $e1.fingerprint \neq e2.fingerprint$.

Invariant 3 (No-orphan edges): After the full pipeline, every edge in `kg_edges` whose endpoints reference entities that survived the Phase 1 merge (i.e., were not tombstoned) satisfies:

$$\forall e \in kg_edges : e.source_id \in kg_entities \wedge e.target_id \in kg_entities$$

Precondition: The pipeline Assumes that no edge references an entity ID that was tombstoned during Phase 1. Tombstoned entities are excluded from the redirect map; edges referencing them require caller-side cleanup. This precondition holds in the evaluation scenarios (§7.1–§7.3).

Theorem 1: The three-phase pipeline preserves Invariant 3 under the precondition that no edge references a tombstoned entity ID.

Proof sketch:

Phase 2 produces a function $R: loser_id \rightarrow winner_id$ (the redirect map) defined on exactly those entity IDs that survived Phase 1 and were merged away. By Invariant 2, every winner ID is a canonical entity ID.

Phase 3 applies R to each edge endpoint before writing. For any edge e whose endpoints survived Phase 1: - If $e.source_id \notin dom(R)$ and $e.target_id \notin dom(R)$, both endpoints were not merged away. By construction of Phase 2, they are already canonical. The invariant holds. - If $e.source_id \in dom(R)$, the rewritten source is $R(e.source_id) = winner_id$, which by Invariant 2 is canonical. Similarly for target.

Therefore every endpoint in the written edge references a canonical entity ID, under the precondition that no edge references a tombstoned entity ID. $\square$

Corollary 1 (Unconditional no-orphan): The no-orphan invariant holds unconditionally — no precondition required.

Proof: The orphan guard (`crdt_projection.py` lines 495–501) filters the merged edge state to only include edges where both `source_id` and `target_id` belong to `canonical_ids` — the set of Phase 2 winner entity IDs (i.e., `set(canonical.keys())` where `canonical` is the output of `entity_dedup_via_crdt`). Phase 2 winners are a subset of Phase 1 survivors (Phase 2 deduplicates Phase 1 output by fingerprint). Since `_apply_entities` iterates `canonical.items()` and INSERTs one row per key (`crdt_projection.py` lines 402–430), `canonical_ids` is exactly the set of `entity_id` values in `kg_entities`. Therefore every edge in the output satisfies Invariant 3 by construction. This guard catches not only merged-away entities (handled by the redirect map) but

also tombstoned entities and references to entity IDs that never had any add operation. The guard is defense-in-depth: Theorem 1 proves the common case (merged-away entities) via the redirect map; the orphan guard handles all remaining cases unconditionally. $\square$

Theorem 2 (Canonical-Id Monotonicity): Let F be a fingerprint group with $|F| \geq 2$. The rule $\text{winner} = \max(F)$ is the unique monotone deterministic selection satisfying:

- (i) $\text{winner}(F) \in F$ (no resurrection — the winner must be a survivor).
- (ii) $|F| = 1 \implies \text{winner}(F) = F$ (singletons are already canonical).
- (iii) $F' \supseteq F \implies \text{winner}(F') \geq \text{winner}(F)$ (monotonicity under re-import — adding more concurrent representations of the same concept cannot lower the canonical ID).

Any other monotone rule either violates (iii) or is non-deterministic.

Proof: We prove by induction on $|F|$ that $\max(F)$ is the unique rule satisfying (i)–(iii).

Base case: $|F| = 1$. By (ii), $m(F) = F = \max(F)$. Yes

Inductive step: $|F| \geq 2$. Let $x = \max(F)$. Since $F \supseteq \{x\}$, by (iii) $m(F) \geq m(\{x\})$. By the base case, $m(\{x\}) = x$. Therefore $m(F) \geq x = \max(F)$. By (i), $m(F) \in F$, so $m(F) \leq \max(F)$. Combining: $m(F) = \max(F)$. Yes

This proves both existence ($\max$ satisfies (i)–(iii)) and uniqueness (any rule satisfying (i)–(iii) must equal $\max$). The proof uses only conditions (i)–(iii); no additional assumptions are needed. $\square$

Empirical verification: `TestMigrationPreservesCanonical` (`test_pipeline.py` lines 799–828) validates this property: when a re-imported entity (ID 3, same fingerprint as ID 5) joins an existing canonical (ID 5), the rule $\max(5, 3) = 5$ preserves the pre-migration winner. An outsider (ID 7, different fingerprint) coexists independently. This confirms that $\max(\text{id})$ is the correct choice: it raises the floor on canonical ID rather than letting re-imported or concurrent outsiders dictate the winner.

5.3 Convergence

Theorem 3 (Determinism over operation set): Given a fixed operation $\log O_E \cup O_{Ev}$, the three-phase pipeline produces a unique canonical state (Σ, σ'_{Ev}) regardless of the order in which operations are processed. All replicas that apply the same operation set reach the same output. (This is a sufficient condition for CRDT convergence under exact-broadcast delivery; delivery-order independence under arbitrary asynchrony is not modeled here.)

Proof:

We prove convergence by showing that each phase is a deterministic function of its input, and that the composition is therefore deterministic with respect to the operation set.

Definitions. Let $\sigma_E = \text{merge_entity_ops}(O_E)$ denote the Phase 1 output (merged entity state). Let $(\Sigma, R) = \text{entity_dedup_via_crdt}(\sigma_E)$ denote the Phase 2 output (canonical entities and redirect map). Let $\sigma_{Ev} = \text{merge_edge_ops}(O_{Ev})$ denote the merged edge state. Let $\sigma'_{Ev} = \text{redirect_edge_ids}(\sigma_{Ev}, R)$ denote the redirected edge state. The final canonical state is (Σ, σ'_{Ev}) .

Phase 1: Entity merge is a CRDT.

We show that `merge_entity_ops` is commutative, associative, and idempotent on the operation set.

Commutativity: The function groups operations by `entity_id`, then processes each group independently. Within a group, the 2P-Set membership test (code lines 159–164) checks whether any remove causally dominates any add — this is a symmetric property of the `vv_dominates` partial order: `vv_dominates(r, a)` is evaluated pair-wise and the result is independent of the order in which pairs are considered (the `any(... for ... for ...)` comprehension is commutative). The LWW field winner (code lines 171–192) iteratively folds over add operations, replacing the running winner whenever a candidate strictly exceeds it under the order (`vv_dominates`, then timestamp desc, then `agent_id` asc). This is an arg-max with respect to a strict total preorder; arg-max is fold-order-independent, so the result is independent of input ordering.

Idempotence: Applying the same operation set twice produces the same grouped structure. Within each group, the 2P-Set test is a property of the cross-product of adds $\times$ removes — `any(vv_dominates(r, a) for a in adds for r in removes)` is a set-functional whose truth value depends only on the sets, not their multiplicities. The LWW field winner is an arg-max reduction over (`vv_dominates`, then timestamp, then `agent_id`); arg-max is multiplicity-independent — its output depends on the set of admissible candidates, not how many times each appears. Therefore `merge_entity_ops(O) = merge_entity_ops(O  $\cup$  O)`.

Associativity: Since merge output depends only on the operation set (proven by commutativity + idempotence), and set union is associative ($(A \cup B) \cup C = A \cup (B \cup C)$), the merge function is well-defined on sets: `merge_entity_ops(S)` depends only on the set S , not on how S was constructed. Therefore all replicas with the same operation set compute the same merged state.

The pipeline is thus a composition of deterministic functions of the operation set: Phase 1 produces a unique σ_E for any O_E , and Phase 2 produces a unique (Σ, R) for any σ_E . Since each phase is a deterministic function of its input (a reduce over operations, not a binary state merge), the composition is deterministic with respect to the operation set. By the CRDT correctness criteria [1], a convergent merge function produces a unique fixed point for any set of operations. Therefore all replicas computing `merge_entity_ops(O_E)` on the same operation set reach the same σ_E .

Phase 2: Dedup is deterministic.

`entity_dedup_via_crdt` is a pure function of σ_E . It groups entries by fingerprint (a deterministic hash) and selects `max(id)` per group. Given the same σ_E , all replicas compute the same (Σ, R) . Each `entity_id` maps to exactly one fingerprint (by definition of the `by_fingerprint` grouping), so the winner/loser partition is uniquely determined: `max(ids)` selects one winner per group, and all others are losers. Therefore Phase 2 is deterministic and order-independent.

Phase 3: Edge merge is a CRDT.

`merge_edge_ops` satisfies the same commutativity, associativity, and idempotence properties as `merge_entity_ops`, by an identical argument (group by `edge_id`, apply `vv_dominates` as primary comparator, timestamp/`agent_id` as tiebreaker). Therefore all replicas compute the same σ_{Ev} .

Phase 3: Redirect is deterministic and idempotent.

`redirect_edge_ids` is a pure function of σ_{Ev} and R . The redirect map R has the property that its domain contains only loser IDs (never winner IDs, by construction of Phase 2). Therefore:

1. After one application, all loser IDs in edge endpoints are replaced by winner IDs.

2. A second application finds no matches (losers are gone, winners are not keys) and makes no changes.

Hence $\text{redirect}(\text{redirect}(S, R), R) = \text{redirect}(S, R)$ — idempotence. Since σ_{Ev} and R are deterministic outputs of prior phases, σ'_{Ev} is deterministic.

Composition.

The full pipeline is $P(O_E, O_{Ev}) = (\Sigma, \sigma'_{Ev})$ where each component is a deterministic function of the prior. By induction:

- σ_E is deterministic (Phase 1 CRDT).
- $(\Sigma, R) = f(\sigma_E)$ is deterministic (Phase 2, pure function).
- σ_{Ev} is deterministic (Phase 3 edge CRDT).
- $\sigma'_{Ev} = g(\sigma_{Ev}, R)$ is deterministic (redirect, pure function).

Therefore P is a deterministic function of (O_E, O_{Ev}) . All replicas applying the same operation set reach the same canonical state. $\square$

Empirical verification. The convergence properties (commutativity, idempotence, determinism) are verified by the property-based test suite (`test_pipeline.py`, classes `TestMergeCommutativity`, `TestEdgeMergeCommutativity`, `TestPropertyDeterminism`, `TestPropertyRedirectIdempotence`, `TestPropertyNoOrphan`). These tests enumerate all permutations of operation orderings for small op sets and assert that the pipeline output is identical in every case.

Theorem 4 (Projection is lossless up to fingerprint equivalence): Let $\lambda : O \rightarrow (\Sigma, \sigma'_{Ev}) \cup \{\perp\}$ be the per-op projection defined by Phases 1–3. Define the equivalence relation $\sim$ on O by $o_1 \sim o_2 \iff \lambda(o_1) = \lambda(o_2)$. The equivalence classes are organized in a two-level hierarchy:

1. **Entity-LWW class** $C_{e,f}^E$: all adds with the same $(entity_id, field)$. λ picks one LWW winner; others map to $\perp$.
2. **Fingerprint class** C_F^F : all entity ops whose `entity_ids` lie in fingerprint group F . λ picks one canonical id per group; loser-entity ops map to $\perp$ — losing both their LWW winner values *and* all the loser’s ops. Each fingerprint class contains the entity-LWW classes for its member `entity_ids` as sub-classes; the fingerprint layer applies a coarser compression above the LWW layer.
3. **Edge-LWW class** C_e^{Ev} : all edge ops with the same `edge_id`. λ picks one LWW winner; others map to $\perp$. Entity classes and edge classes are disjoint (entity ops write to σ_E , edge ops write to σ'_{Ev}).

Property (lossless-up-to-equivalence): Two operation logs O_1, O_2 produce the same canonical projection $\Lambda(O_1) = \Lambda(O_2)$ if and only if they agree on the winner designation in every equivalence class. In particular:

- $(\Rightarrow)$ λ is constant on each equivalence class — the projection is idempotent under equivalence-class member substitution.
- $(\Leftarrow)$ The canonical state $\Lambda(O)$ retains exactly the hierarchy of equivalence classes plus winner designation per class. Anything finer (loser op values, multiplicity, peer of origin within class) is not retained.

Proof: We construct λ explicitly from Phases 1–3 and characterize its kernel.

($\Rightarrow$) λ is **well-defined on classes**. For entity-LWW classes, λ selects the arg-max winner under $(vv_dominates, ts\ desc, agent_id\ asc)$; for fingerprint classes, λ applies the redirect map; for edge-LWW classes, λ selects the edge arg-max. In each case, λ picks a specific class member (the winner), so $\lambda(o) = \lambda(o')$ for any two ops that share a class and winner designation. Substituting one loser for another within a class does not change the projection.

($\Leftarrow$) $\Lambda(O)$ **determines the class hierarchy and winners**. We show (a) the equivalence classes are exactly determined by the two-level hierarchy of $(entity_id, field)$ within fingerprint groups, plus $edge_id$ — no coarser or finer grouping is consistent with $\Lambda(O)$; (b) winners are identified by Σ 's row contents.

For (a), we prove the contrapositive: if two ops are NOT in the same class (by the above definitions), then $\lambda(o_1) \neq \lambda(o_2)$. There are three cases:

Case 1: Different $(entity_id, field)$. Phase 1 groups ops by $entity_id$, then processes each field independently via `_field_winner`. Two ops for different $(entity_id, field)$ pairs contribute to different keys in σ_E — they cannot produce identical λ outputs because the merge function writes distinct dict entries.

Case 2: Same $(entity_id, field)$ but different fingerprint groups. This cannot occur: each $entity_id$ has a fixed fingerprint (computed at inception, immutable). Two ops for the same $entity_id$ are in the same fingerprint group by definition.

Case 3: Different $edge_ids$. Phase 3 groups edge ops by $edge_id$ via `merge_edge_ops`. Two ops for different $edge_ids$ contribute to different keys in σ'_{Ev} — they cannot produce identical λ outputs.

Therefore the equivalence classes are exactly determined by the hierarchy: entity-LWW classes are nested within fingerprint classes (entity ops), and edge-LWW classes are disjoint from entity classes. No two ops from different leaf classes can produce identical λ outputs (the grouping is not coarser), and no op within a class can be split into a finer equivalence without changing $\Lambda(O)$ (the grouping is not finer).

For (b): Σ 's row for entity e contains the LWW winner's metadata (name, type, description) and the canonical fingerprint. The redirect map R records which $entity_ids$ were merged into which winners. Together, (Σ, R) uniquely determines the winner op in each class: the winner is the op whose values appear in Σ 's row.

The nested cascade. The fingerprint layer acts as a *coarser compression nested above LWW*. If entity e_1 and e_2 share a fingerprint with $e_2 = \max(e_1, e_2)$, then e_1 's LWW winners are discarded entirely — not just individual field values, but the whole $entity_id$ sub-tree. The LWW winner of a loser entity is itself a loser at the fingerprint level. This nesting means the information loss is not the union of two independent compressions, but a hierarchical discard where the coarser layer subsumes the finer one.

Therefore $\Lambda(O)$ is equivalent to $(P(O), w)$ where $P(O)$ is the equivalence class hierarchy and w is the winner-selection per class. The discarded information is exactly $O \setminus W(O)$ up to the grouping. $\square$

Corollary 2 (Information-theoretic bound): Σ is a sufficient statistic for the winning-op set $W(O)$, but not for the full operation log O . An adversary with access only to Σ (and R) can identify which ops won each equivalence class but cannot recover the loser set $O \setminus W(O)$.

Proof: By Theorem 4, $\Lambda(O)$ retains exactly the equivalence class hierarchy $P(O)$ and winner des-

ignation w . For any class C with winner $w \in C$, the projection is invariant under which subset of $C \setminus \{w\}$ actually appeared in O — each of the $|C| - 1$ non-winner elements either was present or was not, and the canonical state is the same either way. The number of distinct operation logs O that produce the same $\Lambda(O)$ is therefore $\prod_C 2^{|C|-1}$ (product over all classes), which grows exponentially in the class sizes. In particular, Σ alone cannot distinguish any two such logs: the loser identities, timestamps, version vectors, and peer-of-origin are all unrecoverable from the canonical state. $\square$

5.4 Open Questions

1. **Description-dependent disambiguation.** The fingerprint distinguishes entities only when their descriptions differ. Two entities with the same `(name, type, description)` will merge even if they represent different real-world concepts — just as the old `(name, type)` grouping merged everything. Richer context (source documents, mention text) could strengthen disambiguation; see Future Work.
2. **Concurrent edge writes during the redirect window (resolved).** If agent A adds edge $X \rightarrow Y$ and agent B simultaneously merges Y into Z , the edge $X \rightarrow Y$ references a loser ID at write time. In production, `resolve_edge_endpoints` (`kg/kg_crdt.py:825`) queries the durable `kg_entity_redirect` table at write time, catching this edge and rewriting it to $X \rightarrow Z$. In the standalone artifact (which lacks the durable table), the orphan guard drops the edge. Both approaches prevent orphans; the production path is strictly stronger (rewrite vs. drop). This interaction is resolved by write-time canonicalization.

Worked example: concurrent edge write during redirect. Consider the scenario from §4.5, extended with a concurrent edge write:

Setup. Entity 42 (“alice”, agent A) and entity 99 (“alice”, agent B) have the same fingerprint. Entity 15 (“bob”) exists. Agent A creates edge $(42 \rightarrow 15)$.

Concurrent write. While the projection is running, agent C writes edge $(42 \rightarrow 23)$ referencing entity 42 (which is about to be merged away).

Projection timeline: 1. Phase 1 merges entities: $\{15, 23, 42, 99\}$ survive. 2. Phase 2 deduplicates: 42 and 99 share a fingerprint $\rightarrow$ redirect map $R = \{42 \rightarrow 99\}$. 3. Phase 3 redirects edges: $(42 \rightarrow 15)$ becomes $(99 \rightarrow 15)$. 4. Agent C’s edge $(42 \rightarrow 23)$ arrives after the redirect map is computed.

Without durable redirect table (standalone artifact): The orphan guard checks if 42 is in `canonical_ids`. It is not (42 was merged away). The edge is dropped. **No orphan, but data loss.**

With durable redirect table (production): `resolve_edge_endpoints` queries `kg_entity_redirect`, finds $R(42) = 99$, and rewrites the edge to $(99 \rightarrow 23)$. **No orphan, no data loss.**

After projection, `kg_entity_redirect` contains:

loser_id	winner_id	reason
42	99	collision

The durable table ensures that any edge referencing 42 — regardless of when it was written — is canonicalized to 99 at INSERT time.

6. Implementation

6.1 System Context

The three-phase pipeline is the knowledge-graph projection layer within **agentic-memory**, a local-first memory system for AI agents. The system stores memories as markdown files (the source of truth) with associated SQLite metadata, full-text search, vector embeddings, and a knowledge graph of named entities and typed edges.

Multi-agent synchronization is one of three deployment modes: single-agent local, multi-agent local (multiple processes sharing a filesystem), and multi-agent remote (via the sync server). The CRDT projection pipeline enables the latter two modes without requiring agents to coordinate at write time.

6.2 Schema

Two pairs of tables implement the separation between operation log and canonical state:

Operation log tables:

```
CREATE TABLE kg_entity_crdt (  
  op_id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  entity_id      INTEGER NOT NULL,  
  agent_id       TEXT     NOT NULL,  
  op             TEXT     NOT NULL CHECK (op IN ('add', 'remove')),  
  version_vector TEXT     NOT NULL,  
  name           TEXT,  
  entity_type    TEXT,  
  description    TEXT,  
  fingerprint    TEXT,  
  timestamp      REAL     NOT NULL  
);
```

```
CREATE TABLE kg_edge_crdt (  
  op_id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  edge_id        INTEGER NOT NULL,  
  source_id      INTEGER NOT NULL,  
  target_id      INTEGER NOT NULL,  
  relation       TEXT     NOT NULL,  
  weight         REAL     NOT NULL DEFAULT 1.0,  
  valid_at       TEXT,  
  agent_id       TEXT     NOT NULL,  
  version_vector TEXT     NOT NULL,  
  timestamp      REAL     NOT NULL  
);
```

Note: This is the production-compatible schema used by the test harness (`test_pipeline.py`), the standalone `crdt_projection.py` smoke test, and the **agentic-memory** system. Reproducibility claims in §6.5 are verified against this schema.

Canonical tables:

```
CREATE TABLE kg_entities (  
  entity_id  INTEGER PRIMARY KEY,  
  name       TEXT      NOT NULL,  
  entity_type TEXT      NOT NULL,  
  mentions   INTEGER DEFAULT 1,  
  fingerprint TEXT,  
  UNIQUE(fingerprint)  
);  
  
CREATE TABLE kg_edges (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  source_id   INTEGER NOT NULL,  
  target_id   INTEGER NOT NULL,  
  relation    TEXT      NOT NULL,  
  weight      REAL      DEFAULT 1.0,  
  valid_at    TEXT  
);
```

The operation log tables are append-only; rows are never deleted (remove ops are tombstones). The canonical tables are the projection output and are fully replaceable on each projection run.

The `entity_id` column in `kg_entities` is used as a stable projection key in the standalone reference implementation. In the production `agentic-memory` schema, this column is named `id` and includes auto-increment; the projection logic is otherwise identical.

Redirect-map persistence:

The redirect map (emitted by Phase 2, `entity_dedup_via_crdt`) is a first-class, persistent data structure — not an ephemeral intermediate. It is materialised into a `kg_entity_redirect` table so that merge decisions survive across projection runs and support the audit-trail, reverse-lookup, and incremental-update uses described in §7.3:

```
CREATE TABLE kg_entity_redirect (  
  loser_id   INTEGER NOT NULL,  
  winner_id  INTEGER NOT NULL,  
  reason     TEXT      DEFAULT 'collision',  
  created_at TEXT      DEFAULT (datetime('now')),  
  PRIMARY KEY (loser_id, winner_id)  
);
```

`entity_dedup_via_crdt` emits the redirect map (a `{loser_id: winner_id}` dict); a `persist_entity_redirects` step writes each `(loser_id, winner_id)` pair (with an optional reason, e.g. 'collision') into `kg_entity_redirect`. A companion `resolve_entity_id` query then resolves a (possibly merged-away) entity ID to its canonical winner by walking the redirect table. This persistence is what makes the redirect map a durable artifact rather than a per-run in-memory value.

6.3 Reference Implementation

The following code is a standalone extraction from the production system, stripped of agent-specific logic. Standard-library imports (`dataclasses`, `typing`, `sqlite3`) and helper functions (`_serialise_vv`, `vv_dominates`) are defined elsewhere in the file; only the core pipeline phases are shown here. This is the implementation verified by the test suite in `paper_pipeline/test_pipeline.py`.

```
# --- Data structures ---

@dataclass
class EntityOp:
    entity_id: int
    agent_id: str
    op: str # "add" / "remove"
    version_vector: Dict[str, int] = field(default_factory=dict)
    name: str = ""
    entity_type: str = ""
    description: str = ""
    fingerprint: str = "" # computed at inception, immutable
    timestamp: float = 0.0

@dataclass
class EdgeOp:
    edge_id: int
    source_id: int
    target_id: int
    relation: str = "related_to"
    weight: float = 1.0
    valid_at: Optional[str] = None
    agent_id: str = ""
    version_vector: Dict[str, int] = field(default_factory=dict)
    timestamp: float = 0.0

# --- Fingerprint computation ---

def compute_fingerprint(name: str, entity_type: str, description: str = "") -> str:
    """Compute inception fingerprint for entity identity.

    Same fingerprint → same entity (dedup with max(entity_id) tiebreaker).
    Different fingerprint → different entity (coexist, even if name+type match).
    """
    def canonical(s: str) -> str:
        s = unicodedata.normalize("NFKC", s)
        s = "".join(c for c in s if unicodedata.category(c) not in ("Cf", "Cc", "Co"))
        return " ".join(s.lower().strip().split())
```

```

payload = f"{canonical(name)}|{canonical(entity_type)}|{canonical(description)}"
return hashlib.sha256(payload.encode("utf-8")).hexdigest()

# --- Phase 1: Entity CRDT merge ---

def merge_entity_ops(ops: Iterable[EntityOp]) -> Dict[int, Dict[str, Any]]:
    """Merge entity ops using 2P-Set membership + LWW per field.

    Returns {entity_id: {tombstone, name, entity_type, description}}.
    """
    by_entity: Dict[int, List[EntityOp]] = {}
    for op in ops:
        by_entity.setdefault(op.entity_id, []).append(op)

    result: Dict[int, Dict[str, Any]] = {}

    for entity_id, ops_for_entity in by_entity.items():
        sorted_ops = sorted(
            ops_for_entity,
            key=lambda o: (o.timestamp, _serialise_vv(o.version_vector)),
        )

        adds = [o for o in sorted_ops if o.op == "add"]
        removes = [o for o in sorted_ops if o.op == "remove"]

        if not adds:
            continue

        # 2P-Set: tombstoned if any remove causally follows any add.
        is_tombstoned = any(
            vv_dominates(rem_op.version_vector, add_op.version_vector)
            for add_op in adds
            for rem_op in removes
        )
        if is_tombstoned:
            continue

        # LWW per metadata field: uses causal partial order (vv_dominates).
        # If one op's version vector dominates another's, it wins.
        # Truly concurrent ops (neither dominates) fall back to
        # (timestamp desc, agent_id asc) - a proper total order.
        def _field_winner(field: str) -> str:
            candidates = [o for o in adds if getattr(o, field, "")]
            if not candidates:
                return ""
            winner = candidates[0]
            for candidate in candidates[1:]:

```

```

        if vv_dominates(
            candidate.version_vector, winner.version_vector
        ):
            winner = candidate
        elif not vv_dominates(
            winner.version_vector, candidate.version_vector
        ):
            if (
                candidate.timestamp > winner.timestamp
                or (
                    candidate.timestamp == winner.timestamp
                    and candidate.agent_id < winner.agent_id
                )
            ):
                winner = candidate
        return str(getattr(winner, field))

# Fingerprint: immutable at inception; take from first add op that has one.
# All adds for the same entity_id carry the same fingerprint.
fp = ""
for a in adds:
    if a.fingerprint:
        fp = a.fingerprint
        break

result[entity_id] = {
    "tombstone": False,
    "name": _field_winner("name"),
    "entity_type": _field_winner("entity_type"),
    "description": _field_winner("description"),
    "fingerprint": fp,
}

return result

# --- Phase 2: Entity dedup + redirect map ---

def entity_dedup_via_crdt(
    merged_state: Dict[int, Dict[str, Any]],
) -> Dict[str, Any]:
    """Resolve name-collisions; produce redirect map.

Groups surviving entities by their inception fingerprint.
Returns {"merged_state": ..., "redirects": {loser_id: winner_id}}.
    """

    by_fingerprint: Dict[str, List[int]] = {}
    for entity_id, info in merged_state.items():

```

```

    if info.get("tombstone"):
        continue
    fp = info.get("fingerprint", "")
    if not fp:
        # Legacy: compute from metadata (backfill at projection time)
        fp = compute_fingerprint(
            info.get("name", ""),
            info.get("entity_type", ""),
            info.get("description", ""),
        )
        info["fingerprint"] = fp
    by_fingerprint.setdefault(fp, []).append(entity_id)

deduped: Dict[int, Dict[str, Any]] = {}
redirects: Dict[int, int] = {}

for _fp, ids in by_fingerprint.items():
    if len(ids) == 1:
        deduped[ids[0]] = merged_state[ids[0]]
        continue

    winner_id = max(ids) # deterministic LWW tiebreaker
    deduped[winner_id] = merged_state[winner_id]
    for loser_id in ids:
        if loser_id != winner_id:
            redirects[loser_id] = winner_id

return {"merged_state": deduped, "redirects": redirects}

# --- Phase 3: Edge redirect + projection ---

def redirect_edge_ids(
    edge_state: Dict[int, Dict[str, Any]],
    redirects: Dict[int, int],
) -> Dict[int, Dict[str, Any]]:
    """Rewrite edge endpoints through the redirect map.

Idempotent: applying it twice is equivalent to applying once,
because winner IDs never appear as keys in the redirect map.
    """

    if not redirects:
        return edge_state

    remapped: Dict[int, Dict[str, Any]] = {}
    for edge_id, info in edge_state.items():
        new_info = dict(info)
        if new_info["source_id"] in redirects:

```



```

        new_info["source_id"] = redirects[new_info["source_id"]]
    if new_info["target_id"] in redirects:
        new_info["target_id"] = redirects[new_info["target_id"]]
    remapped[edge_id] = new_info
return remapped

# --- Edge merge (add-only, LWW on weight/valid_at) ---

def merge_edge_ops(ops: Iterable[EdgeOp]) -> Dict[int, Dict[str, Any]]:
    """Merge edge ops using causal dominance (vv_dominates) with timestamp/agent tiebreak.

    The correct CRDT merge: if one op's version vector causally dominates
    another's, it wins. Only truly concurrent ops (neither dominates) fall
    back to (timestamp desc, agent_id asc) tiebreak. This preserves causal
    ordering guarantees and matches the production implementation in
    ``kg/kg_crdt.py``.

    Corrigendum (2026-07-16): earlier versions used vv_sum (summing all
    component clocks) as the primary sort key. That conflates concurrent
    ops with different component-wise clocks and is replaced by vv_dominates.
    """
    by_edge: Dict[int, List[EdgeOp]] = {}
    for op in ops:
        by_edge.setdefault(op.edge_id, []).append(op)

    result: Dict[int, Dict[str, Any]] = {}
    for edge_id, ops_for_edge in by_edge.items():
        winner = ops_for_edge[0]
        for candidate in ops_for_edge[1:]:
            if vv_dominates(candidate.version_vector, winner.version_vector):
                winner = candidate
            elif not vv_dominates(winner.version_vector, candidate.version_vector):
                if (
                    candidate.timestamp > winner.timestamp
                    or (
                        candidate.timestamp == winner.timestamp
                        and candidate.agent_id < winner.agent_id
                    )
                ):
                    winner = candidate
        result[edge_id] = {
            "source_id": winner.source_id,
            "target_id": winner.target_id,
            "relation": winner.relation,
            "weight": winner.weight,
            "valid_at": winner.valid_at,
        }

```

```

    return result

# --- End-to-end projection ---

def project_crdt_to_entities(
    conn: AnyConnection,
) -> Tuple[int, int, Dict[int, int]]:
    """Run Phases 1-3 and write to canonical tables.

    Returns (entities_written, edges_written, redirects_map).
    """
    # Phase 1
    entity_ops = _load_entity_state(conn)
    merged = merge_entity_ops(entity_ops)

    # Phase 2
    dedup = entity_dedup_via_crdt(merged)
    canonical = dedup["merged_state"]
    redirects = dedup["redirects"]

    n_entities = _apply_entities(conn, canonical)

    # Phase 3
    edge_ops = _load_edge_state(conn)
    merged_edges = merge_edge_ops(edge_ops)
    if redirects:
        merged_edges = redirect_edge_ids(merged_edges, redirects)
    # Orphan guard: drop edges whose endpoints are not in canonical_ids
    canonical_ids = set(canonical.keys())
    merged_edges = {
        eid: info
        for eid, info in merged_edges.items()
        if info["source_id"] in canonical_ids
        and info["target_id"] in canonical_ids
    }
    n_edges = _apply_edges(conn, merged_edges)

    conn.commit()
    return n_entities, n_edges, redirects

```

6.4 Correctness Enforcement

The implementation includes two layers of defense against orphans:

1. **Write-time prevention (orphan guard):** The orphan guard in `project_crdt_to_entities` (lines 484–489) filters edges against `canonical_ids` before writing, preventing orphans at write time. This is the primary guarantee — it makes the no-orphan invariant structural.

2. **Post-write detection (verify_crdt_consistency):** A defense-in-depth assertion that checks the no-orphan invariant after projection. In the standalone artifact, this runs externally (tests, smoke test); in production (`kg/kg_crdt.py`), it is called from `project_crdt_to_entities`. A violation indicates a bug in the guard logic or a concurrent write not captured in the operation log. The check queries:

```
SELECT source_id FROM kg_edges
WHERE source_id NOT IN (SELECT entity_id FROM kg_entities);
```

Note on column naming: In the standalone reference implementation the queried column is `entity_id` (as written above). In the production schema (§6.2) the same column is named `id`; the query is otherwise identical.

and asserts the result set is empty for both source and target endpoints. This same check is implemented as `verify_crdt_consistency` in `crdt_projection.py` and called from every projection run in production; in the standalone artifact, it is called externally (tests, smoke test) as defense-in-depth.

6.5 Reproducibility

All claims in this paper are verified against the standalone reference implementation. The implementation and test suite are self-contained: standard-library only (`dataclasses`, `typing`, `sqlite3`), no external dependencies.

Artifact location: `paper_pipeline/crdt_projection.py` (implementation), `paper_pipeline/test_pipeline` (51 tests).

Reproduction steps:

```
cd paper_pipeline
python -m pytest test_pipeline.py -v
```

Expected output: 51 tests pass, covering three end-to-end scenarios (basic concurrent creation, three-way creation, post-merge edge addition), five fingerprint-specific scenarios (homonym disambiguation, migration preservation, fingerprint stability, legacy backfill, empty-description degradation), orphan-guard scenarios (dangling edge to a never-created entity, dangling edge after entity supersession), unit tests for each pipeline phase (Phase 1 merge, Phase 2 dedup, Phase 3 redirect), and property-based tests verifying CRDT convergence invariants (commutativity, idempotence, determinism, no-orphan) as formalized in §5.3.

Expected results: The pipeline produces zero orphan edges in all evaluated scenarios. The no-orphan invariant is verified by `verify_crdt_consistency`, which asserts that every `source_id` and `target_id` in `kg_edges` references an `entity_id` present in `kg_entities`.

System requirements: Python 3.8+, SQLite 3. The implementation uses only the Python standard library.

6.6 Production Integration

The pipeline is deployed in production as part of the `agentic-memory` system. The paper’s claims are validated in production by construction through three integration points:

Sync flow (multi-peer reconciliation). When two agents synchronize, `kg/kg_crdt.py:project_crdt_to_entity` runs the full three-phase pipeline on the merged CRDT log. The sync server (`infra/sync_server.py`) invokes this after receiving remote operations, ensuring the canonical graph is updated before any query is served.

Save flow (single-peer writes). When an agent creates or updates an entity, `save/indexers.py:_record_kg_cr` appends the operation to `kg_entity_crdt` or `kg_edge_crdt`. The CRDT log is append-only; the pipeline runs at sync time, not at save time. This separation means agents never coordinate at write time — the pipeline handles reconciliation.

Write-time canonicalization. Both the CRDT projection path (`apply_edge_crdt_to_db`) and the direct-insert path (`kg_db._upsert_edge`) call `resolve_edge_endpoints` (`kg/kg_crdt.py:825`), which queries the durable `kg_entity_redirect` table to canonicalize every endpoint before INSERT. This structurally prevents orphans regardless of which code path performs the write, even under concurrent edge-write-during-redirect scenarios (§5.4, issue 3).

Convergence under cross-peer sync is verified empirically by `eval/verify_kg_sync_convergence.py`, which runs the pipeline across multiple simulated peers with different operation orderings and asserts identical canonical state.

7. Evaluation

7.1 Correctness Under Concurrency

We evaluate the pipeline against three concurrent-write scenarios using the reference implementation. All scenarios were executed in-memory using SQLite; the operation log was seeded to represent the interleaving of writes from two agents. Test execution was verified against the standalone test suite (`paper_pipeline/test_pipeline.py`), which reproduces the same scenarios programmatically.

Scenario 1: Basic concurrent entity creation (Section 4.5). Two agents create “alice” under IDs 42 and 99. Table 2 shows the result.

Metric	Value
Canonical entities	3 (15/bob, 23/charlie, 99/alice)
Redirect map	{42: 99}
Edges after redirect	2, both referencing 99
Orphan edges	0

Table 2: Scenario 1 result — basic concurrent entity creation. The 3 canonical entities include the two pre-existing entities (15, 23) plus the deduplication winner (99).

Scenario 2: Three-way concurrent creation. Agents A, B, and C each create “project:x” under IDs 10, 20, 30. Table 3 shows the result.

Table 3a: Scenario 2 input — three concurrent agents creating the same entity.

Agent	entity_id	timestamp
A	10	100
B	20	200
C	30	300

Metric	Value
Canonical entity	30 (highest ID)
Redirect map	{10: 30, 20: 30}
Edges after redirect	All rewritten to source 30
Orphan edges	0

Table 3: Scenario 2 result — three-way concurrent creation.

Scenario 3: Post-merge edge addition. After Scenario 1 converges, agent D adds edge (42 → 77) referencing the now-merged-away entity 42. When D’s operation log is projected, the redirect map maps 42 → 99, so the resulting edge is (99 → 77). The write to the merged-away ID is transparently corrected with no special handling required from agent D.

All scenarios pass the no-orphan invariant check. The test suite in `paper_pipeline/test_pipeline.py` includes additional cases for empty tables, duplicate preservation, case sensitivity, and whitespace handling — all passing. Five new test scenarios verify the inception-fingerprint identity (§2.5):

Table 4: Fingerprint identity test scenarios and what they verify.

Test class	What it verifies
<code>TestHomonymDisambiguation</code>	Two “alice” entities with different descriptions → different fingerprints → coexist. Case 3 solved.
<code>TestMigrationPreservesCanonical</code>	Re-imported entity (same fingerprint) collapses with original; outsider (different fingerprint) doesn’t steal the slot. Case 2 solved.
<code>TestFingerprintStabilityOnMetadataUpdate</code>	Description change after creation doesn’t change fingerprint.
<code>TestLegacyBackfill</code>	Entities without fingerprints get computed retroactively from metadata.
<code>TestEmptyDescriptionDegradesGracefully</code>	Empty description → same fingerprint as old grouping key → old behavior preserved.

Backwards compatibility: All pre-fingerprint tests (29 of the current 51) pass without modification because seeded entities have `description=""`, making the fingerprint identical to the old (name, type) grouping key.

Convergence under cross-peer sync is verified empirically by `eval/verify_kg_sync_convergence.py`, which runs the pipeline across multiple simulated peers with different operation orderings and asserts identical canonical state. Combined with the 51 paper tests, the total test coverage is 150+ production + paper tests.

7.1.1 Adversarial Testing

The pipeline was additionally tested against 35 adversarial scenarios (`test_adversarial.py`) covering ten categories: fingerprint collision resistance, version-vector overflow, malicious peer behavior, boundary cases, operational resilience, orphan under partition, redirect map consistency, serializability limits, tombstone edge cases, and mixed operations. Key results:

Table 5: Adversarial test scenarios — selected results from 35 tests across 10 categories.

Category	Test	Result
Fingerprint collision	10,000 ops, same fingerprint	Merge completes in <0.1s; no crash, no OOM
Timestamp drift	Peer skew of 10,000x	LWW tiebreaker picks correct winner
Byzantine VV	Disjoint VVs for same operation	Pipeline does not loop; LWW breaks tie
Orphan under partition	5-way merge, 2 peers partitioned	No-orphan invariant holds post-reconnect
Redirect atomicity	100 edges referencing same redirected ID	All rewritten; no partial application
Serializability	100 edges to same redirected source	All rewritten; no partial application
Tombstone + edge	Edge to tombstoned entity	Orphan guard drops edge; invariant holds
VV overflow	Counters at 10^9	Dominance check still correct

Scaling. Wall-clock time grows linearly with N (merge + dedup, no SQLite I/O). At $K=1000$ (realistic workload: 1000 distinct entities), throughput is 271K ops/s at 100K, 247K at 1M, and 138K at 10M — a 1.96x degradation attributable to Python dict overhead at scale, not algorithmic. At $K=10$ (high dedup), throughput is higher (248K→153K) because fewer fingerprint groups reduce dedup overhead. At 575 bytes/op, 10M ops requires ~5.8 GB memory.

Production path with SQLite. The full pipeline (`project_crdt_to_entities`) includes SQLite reads/writes for loading CRDT logs and writing canonical state. Benchmarking the complete path (1K→1M entity ops with edges, $K=10$) shows throughput of 192K→274K ops/s — comparable to the in-memory merge+dedup numbers. SQLite I/O is not the bottleneck: the entity merge (Phase 1) dominates at ~94% of runtime regardless of whether the DB round-trip is included.

Adversarial fingerprint collision is the most significant result: when an adversary forces all operations onto a single fingerprint group, the merge degrades gracefully (slow but correct) rather than exhibiting catastrophic failure. This is a load-bearing engineering claim — the CRDT does not degrade catastrophically under adversarial input.

Unicode canonicalization. The canonicalization pipeline applies three layers: (1) Unicode NFKC normalization (handles non-breaking spaces, ligatures, compatibility equivalents), (2) format-character stripping (zero-width spaces, RTL marks, BOM), and (3) ASCII case folding + whitespace collapse. Smart quotes are preserved as meaningful punctuation. Eight adversarial tests verify this behavior, including `test_nfkc_normalization_covers_unicode_whitespace` and `test_smart_quotes_not_stripped`.

The invariant check itself is enforced in the projection via `verify_crdt_consistency` in `crdt_projection.py`, which runs after every projection:

```
SELECT COUNT(*) FROM kg_edges e
WHERE e.source_id NOT IN (SELECT entity_id FROM kg_entities)
      OR e.target_id NOT IN (SELECT entity_id FROM kg_entities);
```

Comparison with alternative approaches. Table 7 compares the pipeline against two baselines on the same workload (5,000 concurrent entity ops, 500 edges, 50 distinct entities — 99% dedup ratio).

Metric	Naive merge (ID-at-creation)	Redirect-only	Full pipeline (ours)
Canonical entities	5,000	50	50
Semantic duplicates	4,950	0	0
Orphan edges	460	460	0
Redirect map entries	0	4,950	4,950
Edges pointing to canonical IDs	40	40	40
Converges	Yes	Yes	Yes
Overhead vs naive	—	+37%	+36–39%

Table 7: Baseline comparison on 5,000 concurrent entity ops with 50 distinct entities. The naive merge (equivalent to ID-at-creation approaches like Yjs/Automerger) preserves all 5,000 entities including 4,950 semantic duplicates, and 460 edges reference non-canonical entities. The full pipeline deduplicates to 50 canonical entities, redirects all edges, and eliminates all orphans at ~36–39% overhead — dominated by Phase 1 entity merge (~94% of runtime), not by dedup or redirect.

The key insight: **the overhead of content-keyed dedup is dominated by the entity merge (Phase 1), not by the dedup+redirect (Phases 2–3).** Phase profiling on 10K operations shows Phase 1 accounts for ~94% of runtime; Phases 2–3 (dedup, redirect, orphan guard) together account for ~6%. The real cost is Phase 1, which all three approaches share.

7.1.2 Tombstone Scenario

When a concurrent-creation entity is tombstoned in Phase 1 (remove dominates add), two distinct non-canonical entity types exist after merge:

Non-canonical type	Cause	Handled by	Edge fate
Merged-away	Same fingerprint, lower ID	Redirect map (Phase 3)	Rewritten to winner ID
Tombstoned	Remove dominates add	Orphan guard (Phase 3)	Dropped

The redirect map handles merged-away entities (same fingerprint, different ID). Tombstoned entities are *not* in the redirect map — they are simply absent from the canonical set. The orphan guard drops ALL edges referencing non-canonical entity IDs, regardless of *why* the ID is non-canonical

(merged-away or tombstoned). This is why Corollary 1 states the no-orphan invariant holds unconditionally: the orphan guard is a single filter that handles both cases.

Worked example: tombstoned entity with edges. Agent A creates entity 42 (“alice”) and adds edge $(42 \rightarrow 15)$. Agent B later tombstones entity 42 (remove dominates add). At projection time, entity 42 is absent from the canonical set. The orphan guard encounters edge $(42 \rightarrow 15)$, finds that 42 is not in `canonical_ids`, and drops the edge. The no-orphan invariant holds.

Why the redirect map doesn’t cover tombstones. The redirect map records *which ID was merged into which winner* — a many-to-one mapping within the same fingerprint group. A tombstoned entity has no winner; it was removed entirely. Adding tombstoned entities to the redirect map would require mapping them to a “null” target, which has no meaning in the canonical graph. The orphan guard is the correct mechanism: it drops edges to entities that no longer exist, rather than rewriting them to a meaningless target.

Implication for the precondition of Theorem 1. The precondition “no edge references a tombstoned entity ID” is a *caller* responsibility: callers should not write edges to entities they intend to tombstone. If a caller violates this precondition (writes an edge to entity 42, then tombstones 42), the orphan guard silently drops the edge. This is the correct behavior: the edge is semantically meaningless after the entity is removed. The pipeline is not purely additive — it can reduce the edge set — but this reduction is intentional and prevents orphans by construction.

7.2 Orphan Prevention

The central measurable claim is: **the pipeline produces zero orphan edges under concurrent entity creation, provided no edge references a tombstoned entity (the precondition of Theorem 1)**. This is verified by the invariant check in the test suite and by the reference implementation’s output. A naive merge (entity union without dedup+redirect) produces one orphan edge per concurrent creation event — the edge pointing to the merged-away entity. The pipeline eliminates these orphans by construction (write-time endpoint resolution; §4.4).

Tradeoff — orphan guard as silent data loss. The orphan guard (§4.4, lines 495–501) drops edges whose endpoints reference non-canonical entities. This is defense-in-depth that prevents orphans, but it silently discards edges rather than rewriting them. In production, this is the correct behavior: an edge to a tombstoned or never-created entity is semantically meaningless and should not persist. However, the guard means the pipeline is not purely additive — it can reduce the edge set. The paper’s convergence claim (Theorem 3) applies to the operation set, not to the edge table; the orphan guard is a post-merge filter that is outside the formal convergence model.

7.3 Redirect Map Utility

The redirect map is a persistent output of the pipeline, not an ephemeral intermediate. It enables:

- **Audit trails:** recording which entity IDs were merged, at what projection time, and by which winner, supports debugging and compliance review.
- **Reverse lookups:** given a legacy entity ID that was merged away, the redirect map identifies the canonical replacement. This is useful when migrating queries written against the pre-merge ID space.
- **Incremental updates:** a subsequent projection run can re-apply only those redirects that are new (i.e., where the loser ID still has edges pointing to it), avoiding a full re-projection.

7.4 Limitations

1. **Description-dependent disambiguation.** The fingerprint distinguishes entities only when their descriptions differ. Two entities with the same (`name`, `type`, `description`) will merge even if they represent different real-world concepts — just as the old (`name`, `type`) grouping merged everything. Richer context (source documents, mention text) could strengthen disambiguation; see Future Work.
2. **Canonicalization is Unicode-aware.** The canonicalization pipeline applies Unicode NFKC normalization (handles non-breaking spaces, ligatures, compatibility equivalents), strips format characters (zero-width spaces, RTL marks, BOM), and collapses whitespace. Smart quotes (`\u201C/\u201D`) are preserved as meaningful punctuation — `"alice"` and `alice` produce different fingerprints. Semantic variants (“Skadden” vs “Skadden LLP”) remain outside the scope of surface-level canonicalization. Verified by 8 fingerprint collision tests in `test_adversarial.py`.
3. **No support for entity splitting.** If a canonical entity is later determined to represent two distinct real-world entities, the pipeline has no mechanism to split it. Entity splitting requires a new write operation that creates a new entity and re-associates edges, which is outside the scope of the projection pipeline.
4. **Clock skew in LWW tiebreaker.** The LWW tiebreaker uses wall-clock timestamps as the secondary key (after version vector comparison). In practice, clock skew between peers is bounded by NTP, and the version vector dominates the comparison; the timestamp is only reached when version vectors are identical. However, in adversarial clock-skew scenarios, the tiebreaker may be unreliable. Logical clocks (Hybrid Logical Clocks) would eliminate this concern.
5. **Single redirect pass.** The pipeline handles exactly one round of entity dedup per projection. If a merge produces a new entity that collides with an existing entity in a subsequent projection, the redirect is applied at that time. Incremental multi-round merge is not modeled.
6. **False merges under ambiguous descriptions.** The fingerprint is computed from (`name`, `type`, `description`). If two genuinely distinct entities share all three fields — for example, “Java” the programming language and “Java” the Indonesian island, both typed as `concept` with overlapping descriptions — the pipeline will incorrectly merge them. The false merge rate depends on the domain: in agent memory systems where entities are extracted from conversations, the description field typically contains distinguishing context (“programming language for Android development” vs “largest island in Southeast Asia”), making false merges rare. In domains where descriptions are sparse or generic, the rate increases. The framework (Paper 2) addresses this via composite keys (Theorem 4): extending the key with additional fields (e.g., source document, mention context) reduces false merges. Empirical measurement of false merge rates on real knowledge graph dumps is left as future work.

8. Discussion

8.1 When This Approach Is Appropriate

The three-phase pipeline is well-suited for:

- **Local-first multi-agent systems** where agents share entity identity semantics but cannot coordinate at write time.
- **Systems that can tolerate eventual consistency** — the projection converges but may lag behind concurrent writes.
- **Knowledge graphs with stable entity naming** — where `(name, type)` is a reliable equality criterion, as is common in agent memory systems where entities are extracted from conversations and documents.

8.2 When Not Appropriate

The pipeline is not appropriate for:

- **Strongly consistent systems** requiring immediate global visibility of writes.
- **High-contention environments** where many agents write to the same entity simultaneously — the operation log grows without bound and projection cost increases linearly with log size (currently unoptimized).

8.3 Relationship to the Agent Memory Problem

The pipeline was designed for, and is deployed in, a multi-agent memory system where agents accumulate knowledge about people, projects, and concepts through conversation and observation. In that context, the concurrent-creation problem arises naturally: two agents both meet the same person in different sessions, both extract the entity, and both add it to the graph with their own internal ID. The pipeline ensures that when their operation logs are synchronized, the graph is semantically coherent — one entity, all edges, no orphans.

The implementation described in §6.3 is part of the **agentic-memory** system, a local-first memory store for AI agents. The CRDT projection layer is one of three synchronization modes supported by the system: single-agent local, multi-agent local (shared filesystem), and multi-agent remote (via the sync server).

9. Related Work

§2 covers the CRDT foundations (2P-Set, LWW-Register, version vectors) that the pipeline is built on. This section surveys the broader landscape of systems that face the concurrent-entity-creation problem, and positions our approach relative to existing CRDT designs, entity-resolution techniques, and content-addressed storage.

Table 6: Comparison of entity dedup approaches across CRDT and content-addressed systems.

System	Merge Strategy	Entity Dedup	Edge Handling	Canonicity
Yjs [4]	Y.Map + Y.Array CRDTs; per-object ID assigned at creation	Does not collapse concurrent same-name nodes; duplicates persist	Connector edges bind to stable object IDs	No canonical form; concurrent duplicates survive merge

System	Merge Strategy	Entity Dedup	Edge Handling	Canonicity
Automerge [5]	OR-Set, LWW-Register document CRDT; UUIDs for object identifiers	Same as Yjs — UUIDs guarantee uniqueness, not equality	Same as Yjs	No canonical form
Loro [6]	Delta-based CRDT with composable patches; random IDs at creation	Same approach as Automerge	Same as Yjs	No canonical form
ASIF [10]	G-Counter + LWW-Register per field for entity reconciliation in data integration	Yes — deterministic on content hash; LWW per field	N/A (entity attributes only, no graph edges)	Entity-level canonical form only
Syncthing	Block-level content-addressed sync; file-level CRDT merge	Detects file-level duplicates by content hash; no entity-level dedup	Tombstone + garbage collection for deleted files	No canonical entity form; cleanup is post-hoc
CRDT RDF stores	Triple-level CRDT merge	Does not collapse concurrent duplicate subjects; merge preserves both triples verbatim	No edge rewrite	No canonical form
This work	2P-Set membership + LWW-Register per field	Yes — inception-fingerprint-based dedup (not name+type); redirect map	Pre-projection edge endpoint rewrite via redirect map	Strong canonical form; no-orphan invariant

Adjacent prior art. The individual primitives composing our pipeline have precedents in neighbouring fields, and we position the contribution as their specific packaging rather than their invention:

- *Entity resolution and record linkage* (Fellegi–Sunter [2], Cohen et al. [3]) address the problem of identifying records that refer to the same real-world entity across distributed sources. Our inception fingerprint is a content-based blocking key analogous to Fellegi–Sunter’s matching functions, but computed deterministically at creation time rather than via probabilistic scoring.
- *RDF sameAs merging and URI rewriting* (Silk [11], LIMES [8]) perform entity resolution by asserting equivalence between identifiers and rewriting references accordingly. The **redirect** map is the CRDT-projection analogue of a **sameAs** closure, but it is computed from an inception fingerprint rather than lexical/geometric similarity, and is applied at projection

time rather than as a separate reconciliation pass.

- *Graph-CRDT node merge* (Kleppmann’s work on CRDTs for graphs, and WOOT-graph family approaches) addresses how to merge concurrent node/edge updates in a graph. Our Phase 1/3 decomposition reuses the standard 2P-Set + LWW-Register per field, but adds an explicit fingerprint-keyed dedup that those approaches do not separate out as a first-class, persisted structure.
- *Collaborative editing CRDTs* (Yjs [4], Automerge [5], Loro [6]) use ID-at-creation for position stability. Our framework targets a different problem: entity deduplication where multiple peers create semantically identical records independently. Loro [6] is the closest in architecture (delta-state CRDTs with composable patches) but does not address content-keyed dedup.
- *Semantic integration frameworks* (ASIF [10]) extend CRDTs with per-field LWW-Register semantics for entity reconciliation in heterogeneous big-data environments, but focus on attribute consistency rather than graph edge integrity and orphan prevention.
- *Secure Scuttlebutt / retroactive merge* patterns demonstrate that append-only operation logs can be reconciled after the fact without a coordinator; our pipeline adopts the same log-centric, coordinator-free model and extends it with write-time endpoint canonicalization (`resolve_edge_endpoints`) so that the no-orphan property holds at the moment of write, not merely after reconciliation.
- *Content-hash entity identity* (IPFS/IPLD, datalog provenance) uses a deterministic hash as the canonical identifier. Our inception fingerprint plays an analogous role but is computed over `(name, type, description)` at creation to capture *semantic* rather than *byte* identity.

The specific contribution is the *composition*: a persisted redirect map emitted by a fingerprint-keyed dedup phase, consumed as a pre-canonical edge rewrite in a fixed three-phase order, with write-time endpoint resolution as a structural guarantee. Each element has an antecedent; the packaging and the no-orphan-by-construction guarantee are, to our knowledge, novel for the concurrent-entity-creation problem.

10. Conclusion

We presented a three-phase projection pipeline for conflict-free knowledge graph synchronization in multi-agent systems. The pipeline composes established CRDT mechanisms — 2P-Set for entity membership, LWW-Register per metadata field, and version-vector causality — in a specific ordering that produces a knowledge graph satisfying a strong no-orphan invariant: no edge in the canonical table references a non-canonical entity.

The key design choice is the redirect map emitted as an explicit first-class data structure in Phase 2 and consumed as a rewrite rule in Phase 3. This separates the problem of *which* entities are canonical from the problem of *how* to express that canonicity in a connected graph structure. The separation enables audit trails, reverse lookups, and incremental updates, while keeping each phase individually testable and formally characterizable.

We believe the pipeline is broadly applicable to any local-first system where multiple peers independently create entities with shared naming semantics and must synchronize without a central coordinator. The combination of the redirect map with pre-projection edge rewriting is a novel composition of known CRDT primitives, extending patterns found in RDF `sameAs` merging and graph-CRDT node-merge literature (§9). It solves a problem — concurrent entity creation producing orphan edges — that existing systems either prevent by design or correct after the fact.

Acknowledgements

The author thanks the reviewers for their constructive feedback. Conceptual drafting, literature review, and typesetting assistance used publicly available AI assistants and free-tier paid models; all technical claims, equations, and code references were verified manually against the codebase.

The reference implementation is part of the **agentic-memory** system, released under the Apache-2.0 license.

References

- [1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” in *Stabilization, Safety, and Security of Distributed Systems*, vol. 7032 of *Lecture Notes in Computer Science*, Springer, 2011, pp. 386–400. doi: 10.1007/978-3-642-24550-3_29
- [2] I. P. Fellegi and A. B. Sunter, “A Theory for Record Linkage,” *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, Dec. 1969. doi: 10.1080/01621459.1969.10501049
- [3] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, “A Comparison of String Distance Metrics for Name-Matching Tasks,” in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI 2003)*, Acapulco, Mexico, 2003, pp. 73–77.
- [4] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, “Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types,” in *Proceedings of the 15th International Conference on Web Engineering (ICWE 2015)*, vol. 9114 of *LNCIS*, Springer, 2015, pp. 675–678. doi: 10.1007/978-3-319-19890-3_55
- [5] Automerge Contributors, “Automerge: A CRDT Framework for Collaborative Editing,” 2016–present. [Online]. Available: <https://github.com/automerge/automerge>
- [6] Loro Contributors, “Loro: A CRDT Framework for Collaborative Editing with Delta State,” 2023–present. [Online]. Available: <https://github.com/loro-dev/loro>
- [7] L. D. Ibáñez, H. S. Molli, P. Molli, and O. Corby, “Live Linked Data: Synchronising Semantic Stores with Commutative Replicated Data Types,” *International Journal of Metadata, Semantics and Ontologies*, vol. 8, no. 2, art. 119, 2013. doi: 10.1504/IJMSO.2013.056605
- [8] A.-C. N. Ngomo and S. Auer, “LIMES — A Time-Efficient Approach for Large-Scale Link Discovery on the Web of Data,” in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI 2011)*, 2011, pp. 2312–2317. [Online]. Available: <https://www.ijcai.org/Proceedings/11/Papers/385.pdf>
- [9] J. Benet, “IPFS - Content Addressed, Versioned, P2P File System,” arXiv:1407.3561, 2014.
- [10] Lokesh Butra, Siva Kumar Chintham, Ravi Kiran Kodali, Durgaraman Maruthavanan, and Bikesh Kumar, “ASIF: A Semantic Integration Framework for Heterogeneous Big Data Environments,” Preprint, 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.20741093>
- [11] J. Volz, C. Bizer, M. Gaedke, and G. Kobilarov, “Silk — A Link Discovery Framework for the Web of Data,” in *Proceedings of the WWW2009 Workshop on Linked Data on the Web (LDOW*

2009), CEUR Workshop Proceedings, vol. 538, 2009. [Online]. Available: https://ceur-ws.org/Vol-538/ldow2009_paper13.pdf
